

S1 Supplementary Information

S1.1 Dataset Curation and Limitations

This work is based on a meta-analysis of 295 scintillator materials compiled from existing literature. While the models demonstrate a strong ability to fit the training data, it is important to contextualize their development by discussing the inherent limitations of any study based on a literature-compiled dataset. Several key challenges represent potential sources of uncertainty: "label noise" in the lightyield target variable, which stems from different experimental equipment and calibration standards across publications; "measurement heterogeneity" in the input features, where properties like `host_band_gap` may be inconsistently reported as either theoretically calculated or experimentally measured; and "publication bias," which likely skews the training data towards "successful" high-yield materials, thus preventing the model from learning what features predict poor performance. These factors combined highlight a significant "extrapolation risk." Consequently, the model's utility is highest as a tool for interpolating within the "known" data space, and any predictions for novel materials with properties far outside the training set carry a higher degree of uncertainty. It is evident that the distribution of each feature is not normal; hence, a non-linear technique is required to analyze this data rather than a linear model.

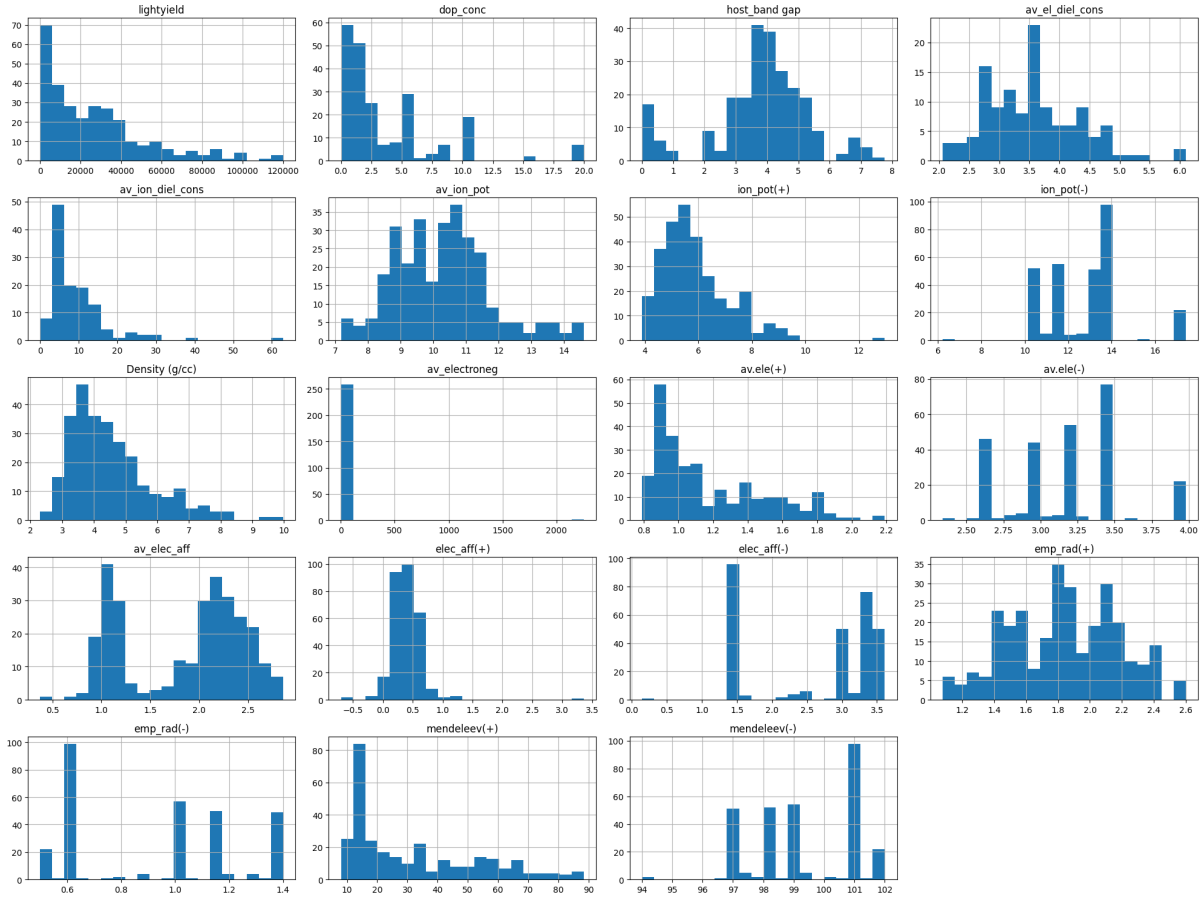


Figure S1: Histograms describing the features' distribution. From top left to bottom right we have the distributions for `lightyield`, `dop_conc` (Dopant concentration), `host_band_gap` (Host band gap), `av_el_di_cons` (Average electronic dielectric constant), `av_ion_pot` (Average ionization potential), `ion_pot(+)` (Cationic ionization potential), `ion_pot(-)` (Anionic ionization potential), `Density (g/cc)`, `av_electroneg` (Average electronegativity), `av.ele(+)` (Cationic electronegativity), `av.ele(-)` (Anionic electronegativity), `av_elec_aff` (Average electron affinity), `elec_aff(+)` (Cationic electron affinity), `elec_aff(-)` (Anionic electron affinity), `emp_rad(+)` (Cationic empirical radius), `emp_rad(-)` (Anionic empirical radius), `mendelev(+)` (Cationic Mendelev number), and `mendelev(-)` (Anionic Mendelev number).

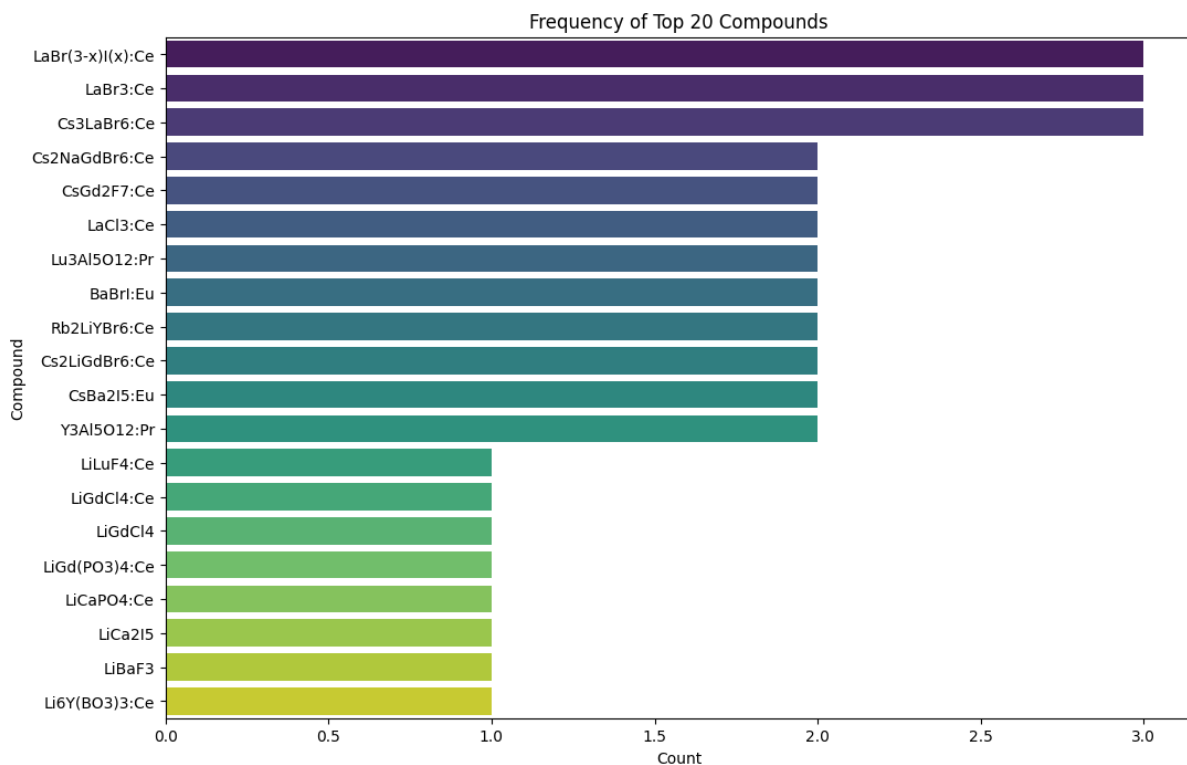


Figure S2: Frequency of Top 20 Compounds

S2 Dataset Curation and Limitations

This work is based on a meta-analysis of 295 scintillator materials compiled from existing literature. While the models demonstrate a strong ability to fit the training data, it is important to contextualize their development by discussing the inherent limitations of any study based on a literature-compiled dataset. Several key challenges represent potential sources of uncertainty: "label noise" in the `lightyield` target variable, which stems from different experimental equipment and calibration standards across publications; "measurement heterogeneity" in the input features, where properties like `host_band_gap` may be inconsistently reported as either theoretically calculated or experimentally measured; and "publication bias," which likely skews the training data towards "successful" high-yield materials, thus preventing the model from learning what features predict poor performance. These factors combined highlight a significant "extrapolation risk." Consequently, the model's utility is highest as a tool for interpolating *within* the "known" data space, and any predictions for novel materials with properties far outside the training set carry a higher degree of uncertainty.

S3 Data Preprocessing, Imputation, and Transformation

The raw dataset (`all_scintillators_cleaned2.csv`) serves as the foundation for the modeling process. A multi-step preprocessing pipeline, detailed in the `train_model.py` script, was applied to ensure data quality and robustness.

S3.1 Data Cleaning

1. **Duplicate Removal:** Any fully duplicate rows were removed.
2. **Target Variable Check:** Samples with no `lightyield` value were dropped, as they cannot be used in a supervised learning workflow.
3. **Feature Pruning:** The columns `av_el_diel_cons` and `av_ion_diel_cons` were dropped entirely due to a high volume of missing values.

S3.2 Median Imputation

Missing data in the remaining numeric feature columns (excluding the `lightyield` target) was handled using **median imputation**. For any feature with missing values, the empty cells were filled with that feature’s median value, calculated from all non-missing samples. This robust method was chosen over mean imputation to prevent skewed distributions and outliers from disproportionately influencing the imputed value.

S3.3 Data Distribution and Log Transformation

The `lightyield` target variable is heavily right-skewed, with many materials clustered at lower light yields and a long tail of high-performing outliers. To stabilize variance and make the distribution more Gaussian (which benefits model training), a **log-transform** (`np.log1p`) was applied to the `lightyield` target. All models were trained to predict this log-transformed value. For final evaluation, predictions are transformed back to the original scale (`np.expm1`) before calculating the R^2 and RMSE metrics against the original, untransformed `lightyield` values.

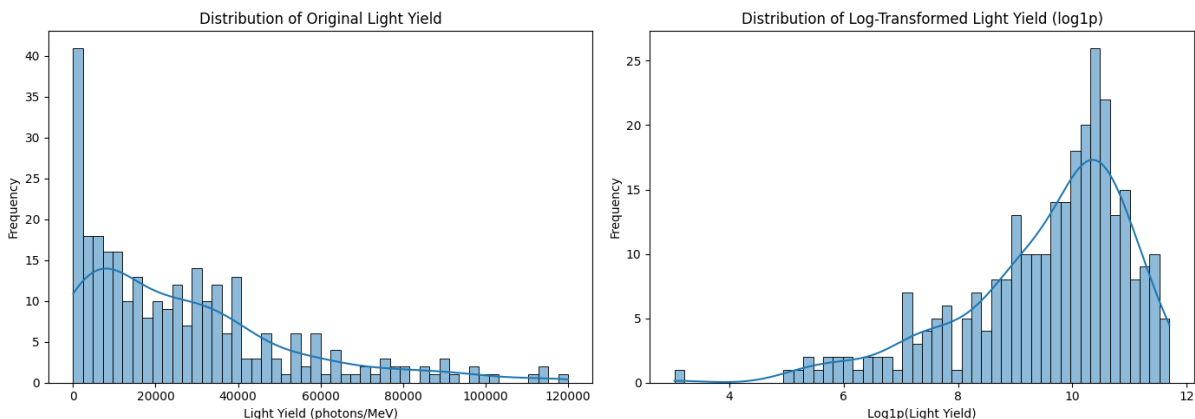


Figure S3: Effect of log transformation on a right-skewed distribution.

S3.4 Physics-Informed Feature Engineering

Following cleaning and imputation, three new physics-informed features were engineered to capture potential interaction effects:

1. `electronegativity_mismatch`: `abs(av.ele(+) - av.ele(-))`
2. `size_mismatch`: `abs(emp_rad(+) - emp_rad(-)) / (emp_rad(+) + emp_rad(-))`
3. `bandgap_doping_ratio`: `host_band_gap / dop_conc` (with a small epsilon added to the denominator to prevent division by zero).

S4 Physicochemical Feature Definitions

The final set of features used for training (after selection) are derived from the elemental properties of the host compound. A brief definition of each base feature is provided below:

S5 Model Hyperparameters

The models were trained using the `scikit-learn` library. The `RandomForestRegressor` was selected as the final model, and its hyperparameters (used in `train_model.py`) are listed below. These were left as the library defaults to avoid overfitting to the relatively small dataset, but could be tuned in future work.

Table S1: Definitions of physicochemical features.

Feature	Definition
dop_conc	Dopant concentration (e.g., Ce, Eu) as reported in the literature.
host_band_gap	The electronic band gap of the host material (eV).
av_ion_pot	Average ionization potential of the constituent atoms.
ion_pot(+)	Average ionization potential for the cationic elements.
ion_pot(-)	Average ionization potential for the anionic elements.
Density (g/cc)	Density of the host material.
av_electroneg	Average Pauling electronegativity of the constituent atoms.
av.ele(+)	Average electronegativity for the cationic elements.
av.ele(-)	Average electronegativity for the anionic elements.
av_elec_aff	Average electron affinity of the constituent atoms.
elec_aff(+)	Average electron affinity for the cationic elements.
elec_aff(-)	Average electron affinity for the anionic elements.
emp_rad(+)	Average empirical radius for the cationic elements (Å).
emp_rad(-)	Average empirical radius for the anionic elements (Å).
mendeleeev(+)	Average Mendeleev number for the cationic elements.
mendeleelev(-)	Average Mendeleev number for the anionic elements.

Table S2: Hyperparameters for the RandomForestRegressor model.

Model	Hyperparameter	Value
RandomForestRegressor	n_estimators	100
	criterion	squared_error
	max_depth	None
	min_samples_split	2
	min_samples_leaf	1
	random_state	42

S6 Supplementary Figures

This section contains additional model diagnostics and data visualizations generated during the training and evaluation process.

S6.1 Feature Correlation Heatmap

This heatmap visualizes the Pearson correlation coefficient between all 16 input features. High correlation (values near +1.0 or -1.0) indicates strong multicollinearity. This can be used to identify redundant features and inform feature selection.

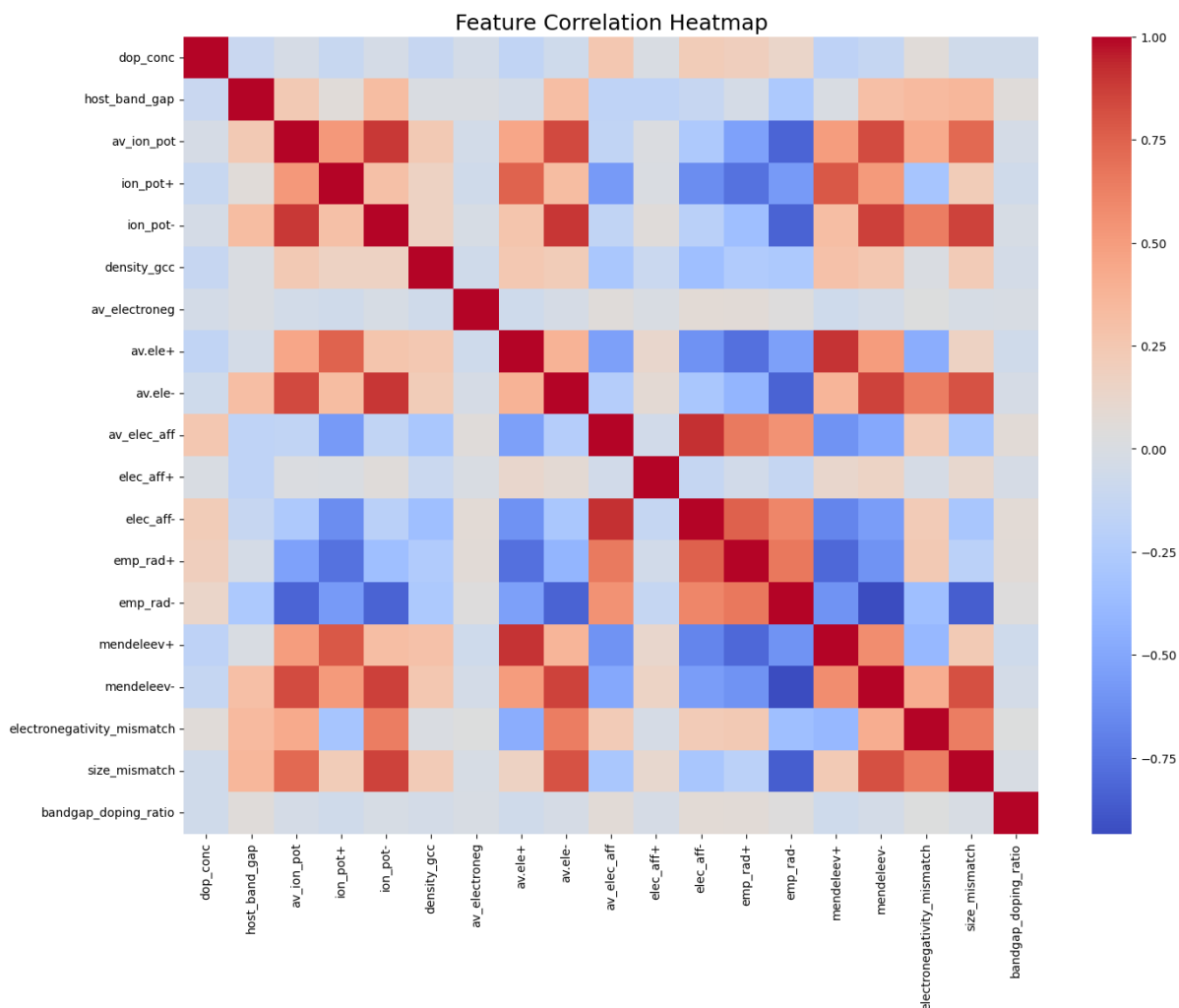


Figure S4: Feature Correlation Heatmap.

S6.2 SHAP Summary Plot

SHAP Values for best model (XGBoost)

One of the disadvantages of the inherent feature importance scores obtained from the GBM model is that the directionality is not apparent i.e. when a particular feature for a light yield sample takes a high value, does that correspond to high or low feature importance score. Moreover, at the test phase, it is not straightforward for traditional white-box, tree-based, machine learning techniques to provide information about, say, the top features driving the prediction. Recently, several techniques such as the **LIME** [4] and **SHAP** [3] methods have been proposed to overcome these limitations. These methods have the ability to interpret feature importance scores from complex training models as well provide

interpretable predictions for a test sample by grounding their reasoning on the top k features for that particular test instance.

In our work, we use the **SHAP (SHapley Additive exPlanations)** method, a unified framework for interpreting predictions, as it was shown in [3] to outperform LIME and they demonstrated that its predictions are better aligned with human intuitions. The SHAP method belongs to the class of **additive feature attribution methods** where a test instance prediction is composed as a linear function of features and satisfies three critical properties comprised of **local accuracy**, **missingness** and **consistency**.

The explicit SHAP regression values come from a game-theory framework [2],[5] and can be computed as:

$$\phi_i = \sum_{S \subseteq Q \setminus \{i\}} \frac{|S|!(|Q| - |S| - 1)!}{|Q|!} [H(S \cup \{i\})(x_{S \cup \{i\}}) - H_S(x_S)]$$

Here, Q represents the set of all d features and S represents the subsets obtained from Q after removing the feature and ϕ is an estimate of the importance of feature i in the model. In order to refrain from undergoing differences to estimate ϕ , the SHAP method approximates the Shapley value by either performing **Shapley sampling** [5] or **quantitative input influence** [1]. A detailed description of the SHAP method for model interpretation is available [3]. We passed our model along with the training set to the SHAP method to obtain importance of features based on Shapley values. This plot, generated using the SHAP (SHapley Additive exPlanations) framework, displays the most important features driving the model’s predictions. Each point represents a single sample, with its color indicating the feature’s value (high or low) and its x-position indicating its impact on the prediction (pushing it higher or lower). This provides global feature importance and insight into the model’s internal logic. We passed our model along with the training set to the SHAP method to obtain importance of features based on Shapley values. Figure S5 highlights the ranked training features based on Shapley values. Moreover, it also provides directionality, meaning when a feature attains ‘high’ or ‘low’ values, the corresponding Shapley values are positive or negative. Positive Shapley values drive the predictions towards higher values of light yield, whereas negative Shapley values influence the predictions to move toward lower values of light yield. From Figure S5, we can observe that when top features such as [e.g., ‘host_band_gap’, ‘density_gcc’] take high values, the corresponding Shapley values are positive driving model prediction to bigger light yield values, whereas when these features take low values (i.e. closer to 0), the corresponding Shapley values are negative. Similarly, when other features such as [e.g., ‘electronegativity_mismatch’, ‘size_mismatch’] are high, the corresponding Shapley values are negative driving the model prediction towards smaller light yield values.

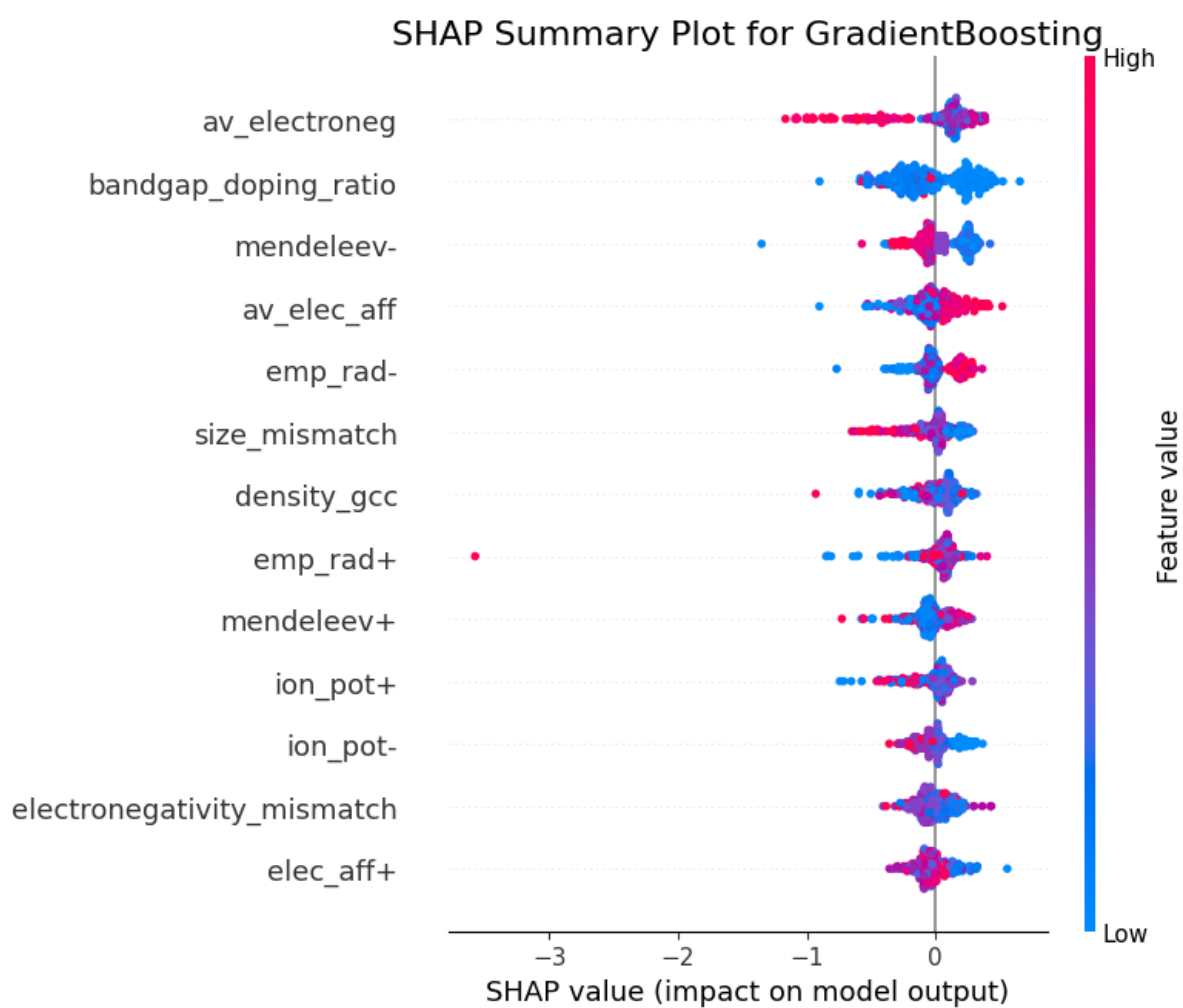


Figure S5: SHAP Summary Plot.

S6.3 Code

```
1 #
2 # 0. SETUP AND IMPORTS
3 #
4 import pandas as pd
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 from sklearn.model_selection import RandomizedSearchCV
9 from sklearn.feature_selection import RFECV
10 from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
11 from sklearn.metrics import mean_squared_error, r2_score
12 import xgboost as xgb
13 import joblib
14 import warnings
15
16 warnings.filterwarnings('ignore')
17 print("Libraries imported successfully.")
18
19 #
20 # 1. DATA LOADING AND INITIAL ASSESSMENT
21 #
22 print("\n--- 1. Loading and Assessing Data ---")
23 try:
24     # Assumes the CSV is in the same directory or a specified path
25     df =
26         pd.read_csv('/content/drive/MyDrive/PET_work/all_scintillators_cleaned2.csv')
27     print("Dataset loaded successfully.")
28 except FileNotFoundError:
29     print("Error: Raw data file 'all_scintillators_cleaned2.csv' not found.")
30     exit()
31
32 # Clean column names for easier access
33 df.columns = df.columns.str.strip().str.lower().str.replace(' ',
34     '_').str.replace('(', '').str.replace(')', '').str.replace('/', '')
35 print("Cleaned column names.")
36
37 #
38 # 2. DATA CLEANING AND PREPROCESSING
39 #
40 print("\n--- 2. Data Cleaning and Preprocessing ---")
41 df_cleaned = df.copy()
42 df_cleaned.drop_duplicates(inplace=True)
43 df_cleaned.dropna(subset=['lightyield'], inplace=True)
44 # Drop columns with a high number of missing values if they exist
45 cols_to_drop_high_missing = ['av_el_diel_cons', 'av_ion_diel_cons']
46 df_cleaned.drop(columns=cols_to_drop_high_missing, inplace=True,
47     errors='ignore')
48 # Impute missing numeric values with the median
49 numeric_cols = df_cleaned.select_dtypes(include=np.number).columns.tolist()
50 numeric_cols.remove('lightyield')
51 for col in numeric_cols:
52     if df_cleaned[col].isnull().any():
53         df_cleaned[col].fillna(df_cleaned[col].median(), inplace=True)
54 print("Data cleaning and imputation complete.")
```



```

53 #
54 # =====
55 # 3. PHYSICS-INFORMED FEATURE ENGINEERING
56 # =====
57 print("\n--- 3. Physics-Informed Feature Engineering ---")
58 epsilon = 1e-6
59 df_cleaned['electronegativity_mismatch'] = np.abs(df_cleaned['av.ele+'] -
60 df_cleaned['av.ele-'])
61 df_cleaned['size_mismatch'] = np.abs(df_cleaned['emp_rad+'] -
62 df_cleaned['emp_rad-']) / (df_cleaned['emp_rad+'] + df_cleaned['emp_rad-']
63 + epsilon)
64 df_cleaned['bandgap_doping_ratio'] = df_cleaned['host_band_gap'] /
65 (df_cleaned['dop_conc'] + epsilon)
66 print("Engineered features created successfully.")
67
68 #
69 # =====
70 # 4. PREPARE DATA FOR MODELING (X AND Y)
71 # =====
72 print("\n--- 4. Preparing Data for Modeling ---")
73 features_to_exclude = ['lightyield', 'scintillator', 'mechanism', 'reference']
74 X = df_cleaned.drop(columns=[col for col in features_to_exclude if col in
75 df_cleaned.columns])
76 X = X.select_dtypes(include=np.number)
77 y = np.log1p(df_cleaned['lightyield']) # Use log1p for stability
78 print(f>Data prepared with {X.shape[1]} initial features.")
79
80 #
81 # =====
82 # 5. DEFINE MODELS AND HYPERPARAMETER GRIDS FOR TUNING
83 # =====
84 models_to_tune = {
85     'RandomForest': {
86         'model': RandomForestRegressor(random_state=42),
87         'params': {
88             'n_estimators': [100, 200, 300], 'max_depth': [10, 20, 30, None],
89             'min_samples_split': [2, 5, 10], 'min_samples_leaf': [1, 2, 4],
90             'max_features': ['sqrt', 'log2', 1.0]
91         }
92     },
93     'GradientBoosting': {
94         'model': GradientBoostingRegressor(random_state=42),
95         'params': {
96             'n_estimators': [100, 200, 300], 'learning_rate': [0.01, 0.05,
97 0.1],
98             'max_depth': [3, 5, 7], 'subsample': [0.8, 0.9, 1.0]
99         }
100     },
101     'XGBoost': {
102         'model': xgb.XGBRegressor(random_state=42,
103 objective='reg:squarederror'),
104         'params': {
105             'n_estimators': [100, 200, 300], 'learning_rate': [0.01, 0.05,
106 0.1],
107             'max_depth': [3, 5, 7], 'subsample': [0.8, 0.9, 1.0],
108             'colsample_bytree': [0.8, 0.9, 1.0]
109         }
110     }
111 }
112 results = {}

```

```

102
103 #
=====
104 # 6. ITERATE, TUNE, AND EVALUATE EACH MODEL
105 #
=====
106 for model_name, config in models_to_tune.items():
107     print(f"\n--- Processing Model: {model_name} ---")
108
109     # Feature Selection with RFECV
110     estimator_for_rfe = config['model']
111     selector = RFECV(estimator=estimator_for_rfe, step=1, cv=5, scoring='r2',
112                     n_jobs=-1)
113     selector.fit(X, y)
114     X_selected = X.loc[:, selector.support_]
115     print(f"RFECV selected {selector.n_features_} features.")
116
117     # Hyperparameter Tuning
118     random_search = RandomizedSearchCV(
119         estimator=config['model'], param_distributions=config['params'],
120         n_iter=30, cv=5, random_state=42, n_jobs=-1, scoring='r2'
121     )
122     random_search.fit(X_selected, y)
123     best_model = random_search.best_estimator_
124     print("Hyperparameter tuning complete.")
125
126     # Evaluation
127     y_pred_log = best_model.predict(X_selected)
128     y_pred_final = np.expm1(y_pred_log)
129     y_actual_final = np.expm1(y)
130     r2 = r2_score(y_actual_final, y_pred_final)
131     rmse = np.sqrt(mean_squared_error(y_actual_final, y_pred_final))
132
133     # Store Results
134     results[model_name] = {
135         'best_model': best_model, 'r2_score': r2, 'rmse': rmse,
136         'selected_features_count': selector.n_features_,
137         'y_pred': y_pred_final, 'y_actual': y_actual_final
138     }
139 #
=====
140 # 7. VISUALIZE AND COMPARE RESULTS (CODE FOR THE FIGURE)
141 #
=====
142 print("\n--- 7. Generating Final Comparison Visualization ---")
143
144 num_models = len(results)
145 fig, axes = plt.subplots(1, num_models, figsize=(6 * num_models, 5.5),
146                         sharey=True)
147 fig.suptitle('Parity Plots: Predicted vs. Actual Light Yield', fontsize=20)
148
149 for i, (model_name, data) in enumerate(results.items()):
150     ax = axes[i]
151     sns.scatterplot(x=data['y_actual'], y=data['y_pred'], alpha=0.7, ax=ax)
152     # Plot the ideal y=x line
153     ax.plot([data['y_actual'].min(), data['y_actual'].max()],
154             [data['y_actual'].min(), data['y_actual'].max()], 'r--', lw=2)
155     ax.set_title(f"{model_name}\n(R2 = {data['r2_score']:.4f})", fontsize=14)
156     ax.set_xlabel('Actual Light Yield (photons/MeV)')
157     if i == 0:
158         ax.set_ylabel('Predicted Light Yield (photons/MeV)')
159     ax.grid(True)

```

```

159
160 plt.tight_layout(rect=[0, 0.03, 1, 0.95])
161 plt.show()

```

References

- [1] A. Datta, S. Sen, and Y. Zick. Algorithmic transparency via quantitative input influence: Theory and experiments with learning systems. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 598–617, 2016. doi: 10.1109/SP.2016.42.
- [2] S. Lipovetsky and M. Conklin. Analysis of regression in game theory approach. *Applied Stochastic Models in Business and Industry*, 17, 2001. ISSN 15241904. doi: 10.1002/asmb.446.
- [3] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf.
- [4] M. T. Ribeiro, S. Singh, and C. Guestrin. "why should i trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, page 1135–1144, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450342322. doi: 10.1145/2939672.2939778. URL <https://doi.org/10.1145/2939672.2939778>.
- [5] L. S. Shapley. *A Value for n-Person Games*, pages 307–318. Princeton University Press, 12 1953. doi: 10.1515/9781400881970-018. URL <https://www.degruyter.com/document/doi/10.1515/9781400881970-018/html>.